



BIOLOGICAL DATA ANALYSIS WITH R

Data Import, Cleaning & Wrangling

Follow-up Lecture — From Raw Files to Analysis-Ready Biological Data

Hands-on session — import · clean · dplyr · tidyr



```
> df <- read.csv("culture_od600.csv")
> df <- df %>% filter(!is.na(od600))
> df %>% group_by(treatment) %>%
+   summarise(mean_od = mean(od600))
```

Where We Left Off

Last session added tools for categorical data and reusable, decision-driven code

1

Factors

`factor()`, `levels()` and `table()` for categorical biological data

2

Control Structures

`if / else`, `for` and `while` loops to automate repeated logic

3

Custom Functions

`function()`, arguments and `return()` to package reusable code

4

`supply()`

Applying a custom function across every element of a vector

Today we bring in real files — importing, cleaning, and reshaping biological datasets so they're ready for analysis

Learning Objectives

1

Import real datasets

Read CSV, tab-delimited text and Excel files into R as data frames

2

Clean messy data

Detect and handle missing values, duplicate rows and outliers

3

Wrangle with dplyr

Filter, select, mutate, arrange, group and summarise data

4

Reshape with tidyr

Convert between wide and long formats using pivot_longer/pivot_wider



```
> df <- read.csv("culture_od600.csv")
> df <- df[!duplicated(df), ]

> library(dplyr)
> df %>% filter(od600 < 5) %>%
+   arrange(desc(od600))

> library(tidyr)
> wide %>% pivot_longer(
+   cols = starts_with("Day"),
+   names_to = "day", values_to = "od600")
```

Installing the Required Packages

readxl, dplyr and tidyr are not part of base R — install each once per machine, then load with library() every session

```
install.packages("readxl")
```

Reads Excel .xlsx / .xls files into R

```
install.packages("dplyr")
```

Provides filter, select, mutate, group_by, summarise

```
install.packages("tidyr")
```

Provides pivot_longer / pivot_wider for reshaping data



```
# Run once (skip if already installed)
install.packages("readxl")
install.packages("dplyr")
install.packages("tidyr")

# Run at the start of every session
library(readxl)
library(dplyr)
library(tidyr)
```

Reading Files into R

Three functions cover almost every biological dataset format you'll encounter

`read.csv()`

Reads a comma-separated (.csv) file into a data frame

`read.table()`

Reads a delimited text file — specify sep for tab ("t") or space

`read_excel()`

From the readxl package — reads a named sheet from an .xlsx file

`getwd()` / `setwd()`

Check or set the working directory R reads relative paths from

```
> df <- read.csv("culture_od600.csv")
> head(df)

> expr <- read.table(
+   "gene_expression.txt",
+   header = TRUE, sep = "\t")

> library(readxl)
> meta <- read_excel(
+   "patient_metadata.xlsx",
+   sheet = "Metadata")
```

Handling Missing Values (NA)

culture_od600.csv includes a few missing OD600 readings — a realistic instrument/recording gap

`is.na()`

Returns TRUE/FALSE for each element — is it missing?

`sum(is.na(x))`

Counts how many missing values a column contains

`na.omit()`

Removes any row containing at least one NA

`complete.cases()`

Logical vector flagging rows with no missing values

```
> sum(is.na(df$od600))
[1] 3

> df_clean <- na.omit(df)
> nrow(df); nrow(df_clean)
[1] 38
[1] 35

> df %>% filter(!is.na(od600))
```

Duplicates

The same dataset also has two duplicate rows and one implausible OD600 reading

`uplicated()`

Flags rows that are exact repeats of an earlier row

`unique()` / `distinct()`

Removes duplicate rows (`distinct()` is the dplyr version)



```
> sum(duplicated(df))  
[1] 2  
> df <- df[!duplicated(df), ]
```

dplyr Basics: filter, select, mutate, arrange

Four verbs cover most everyday data wrangling — chained together with the pipe `%>%`

`filter()`

Keep only rows matching a condition

`select()`

Keep only the specified columns

`mutate()`

Add or modify a column, often from existing ones

`arrange()`

Sort rows by one or more columns

```
> library(dplyr)
> df %>%
+   filter(treatment == "Treated") %>%
+   select(sample_id, day, od600) %>%
+   mutate(od600_log = log(od600)) %>%
+   arrange(desc(od600))
```

Reshaping Data: `pivot_longer()` & `pivot_wider()`

`growth_wide.csv` stores one OD600 reading per day as separate columns — `tidyr` reshapes it for analysis and plotting

`pivot_longer()`

Collapses multiple columns into key–value pairs
(wide → long)

`pivot_wider()`

Spreads key–value pairs back into separate
columns (long → wide)

`names_to` / `values_to`

Names the new columns created by
`pivot_longer()`

`starts_with()`

Selects columns by a shared name prefix (e.g.,
"Day")

```
> library(tidyr)
> wide <- read.csv("growth_wide.csv")
> long <- wide %>% pivot_longer(
+   cols = starts_with("Day"),
+   names_to = "day",
+   values_to = "od600")

> long %>% pivot_wider(
+   names_from = day,
+   values_from = od600)
```

Key Takeaways

Import

`read.csv()` · `read.table()` · `read_excel()` — every file type into a data frame

Clean

`is.na()` · `duplicated()` · `IQR()` — find and remove bad data before analysis

Wrangle

`filter()` · `mutate()` · `group_by()` · `summarise()` · `left_join()` — the dplyr toolkit

Reshape

`pivot_longer()` · `pivot_wider()` — switch between wide and long formats

Remember: always clean (missing values, duplicates, outliers) before you wrangle or reshape — summarising dirty data produces confidently wrong answers.

COMING UP NEXT

Exploratory Data Analysis & Visualization

Descriptive statistics, base R plotting, and ggplot2 — turning cleaned biological data into insight through summary numbers and visuals.

- Descriptive statistics: mean, median, variance, SD, quartiles
- Base R plots: histograms, boxplots and scatterplots
- Introduction to ggplot2 and the grammar of graphics

Thank You

Dr. M. R. D. S. Nitish Venkateswarlu · Department of Biotechnology · Parul Institute of Technology